Queue Shift: Updating Classifiers Under a Staffing Budget

Gaurav Sood

August 2026

Abstract

When a classifier routes cases to specialized queues, a model update changes both prediction accuracy and the distribution of work. Backward-compatibility methods count negative flips, which do not identify how much work must move between queues. I define queue shift as half the L_1 distance between incumbent and proposed queue totals, and choose the batch assignment that maximizes candidate-model probability subject to budgets on queue shift and on expected negative flips. Without the flip budget, or with a flip penalty, the problem is a minimum-cost flow with an integral optimum; with a hard flip budget it is a small integer program. At the queue shift and expected flips of any comparison assignment, including any point on a negative-flip-rate interpolation path, the solution has at least the comparison's expected accuracy under the candidate's probabilities. The flow optimum is also a per-queue price rule, which links the method to online allocation and gives the natural low-cost alternative. Simulations then map where the constraint matters. Holding queue totals fixed costs little when an update mostly reorders cases and a great deal when it moves real volume between queues. Learned prices nearly match exact assignment in accuracy but miss the movement budget in most batches.

1 The operational problem

Suppose a support organization sends predicted billing tickets to a billing team, technical tickets to a technical team, and so on. It replaces its routing classifier after a new feature or model improves accuracy on later held-out data. The raw update may also change the number of cases sent to each team. Even when total demand is fixed, managers may need to move capacity across queues.

This creates a decision after prediction. The organization has already chosen a better candidate model. It now needs to decide which queue receives each case, balancing the candidate's accuracy signal against a limit on workload redistribution. Retraining the model to reduce negative flips addresses a related concern, but it does not directly control queue totals.

Research on prediction churn and backward-compatible updates measures changes to individual decisions, including cases on which a formerly correct prediction becomes wrong (Milani Fard et al., 2016; Bansal et al., 2019; Srivastava et al., 2020; Träuble et al., 2021; Jiang et al., 2022). That protection can matter when people depend on particular decisions. The staffing problem is different because cases can trade queues while every team receives the same amount of work.

Let $i = 1, \ldots, n$ index cases and $k = 1, \ldots, K$ index queues. The incumbent sends case i to a_i, producing load

$$m_k = \sum_{i=1}^{n} \mathbf{1}\{a_i = k\}.$$

For a proposed assignment z, let $n_k(z)$ be its load in queue k. Define

$$\mathcal{S}(z; m) = \frac{1}{2} \sum_{k=1}^{K} |n_k(z) - m_k| = \sum_{k=1}^{K} [n_k(z) - m_k]_+.$$

The equality follows because old and new loads both sum to n. Queue shift is the smallest number of workload units that must cross queue boundaries to reconcile the two load vectors. A value of zero preserves every queue total, even if individual cases exchange places.

That distinction is why negative-flip rate (NFR) does not identify the staffing cost. Consider three cases with true labels $(1, 2, 1)$ and incumbent assignments $(1, 1, 2)$. Compare proposed assignments $(2, 2, 2)$ and $(2, 1, 1)$. Each proposal has one correct prediction, one negative flip, one positive flip, and two changed case labels. The first changes queue loads from $(2, 1)$ to $(0, 3)$, so its queue shift is two. The second keeps loads at $(2, 1)$, so its queue shift is zero. Accuracy, NFR, and total label churn are identical, but the staffing implications differ.

Counting cases treats a billing ticket and a fraud investigation as the same amount of work. Staffing cares about hours or money. Let $c_k \geq 0$ be the price of one case in queue k: one for a case count, average handle time for hours, average cost per case for money. The cases are the currency and c_k is the exchange rate. The workload shift

$$\mathcal{S}_c(z; m) = \sum_{k=1}^{K} c_k \, [n_k(z) - m_k]_+$$

prices the work added to queues beyond their incumbent loads. Work removed from one queue earns no credit against work added to another, because staff are not interchangeable across queues. With $c_k = 1$, $\mathcal{S}_c = \mathcal{S}$. Per-queue averages ignore variation in handling time within a queue and differences in skill; the model keeps predicted class as the unit that determines a staffed queue.

2 Assignment under movement and flip budgets

Let p_{ik} be the candidate model's probability that case i belongs in queue k. For any assignment z, define its average candidate score as

$$V_p(z) = \frac{1}{n} \sum_{i=1}^{n} p_{i,z_i}.$$

Queue totals are not the only thing an update disturbs. Each case i that leaves its incumbent queue a_i is a changed decision, and it is a negative flip exactly when its true class is a_i. Give each case a nonnegative weight w_i and define the flip load

$$F_w(z) = \sum_{i=1}^{n} w_i \, \mathbf{1}\{z_i \neq a_i\}.$$

Two weights matter. With $w_i = 1$, F_w counts changed labels. With $w_i = p_{i,a_i}$, F_w is the expected number of negative flips under the candidate's probabilities, because a moved case is a negative flip with probability p_{i,a_i}. Neither weight uses labels. Given an integer movement budget B and a flip budget C, Queue Shift solves

$$z^*(B, C) \in \arg\max_{z} V_p(z) \quad \text{subject to} \quad \mathcal{S}(z; m) \leq B, \quad F_w(z) \leq C. \tag{1}$$

Setting $C = \infty$ gives the movement-only problem. The dominance result and error bounds below hold with the priced workload shift $\mathcal{S}_c$ in place of $\mathcal{S}$; the queue-price result needs the flow form. Equal prices keep that form, with the overflow capacity B/c; unequal prices turn the movement budget into a weighted side constraint, solved, like the flip budget, as an exact integer program.

The movement-only problem is a minimum-cost flow. A source sends one unit through each case node. Every case connects to every queue with cost $1 - p_{ik}$. Queue k sends up to m_k units directly to the sink at no movement cost. Additional units pass through one shared overflow node, whose edge to the sink has capacity B. Integer supplies and capacities imply an integral optimum, so the linear program assigns every case to exactly one queue. A flip penalty keeps this structure: maximizing $V_p(z) - \lambda F_w(z)/n$ only adds λw_i to the cost of every arc from case i to a queue other than a_i. A hard flip budget does not. It adds one constraint that couples all cases, the linear relaxation can be fractional, and the problem must be solved as an integer program. With HiGHS at a zero optimality gap, a 300-case, 15-queue batch takes well under a second and a 2,000-case batch a few seconds; the penalty form solves the same 2,000-case batch in under a tenth of a second. The two forms agree where they meet. A penalty solution z_λ is optimal for the hard problem at $C = F_w(z_\lambda)$, since any assignment with no more flip load and a higher score would also have a higher penalized objective.

The transport construction itself is established. Probability-based label allocation and classification under capacity constraints already use transport or mathematical programming (Tai et al., 2021; Rabkin et al., 2024), and predict-then-optimize methods incorporate downstream decisions into predictive systems (Elmachtoub and Grigas, 2022). Work on prediction churn asks which updates change individual decisions (Träuble et al., 2021; Watson-Daniels et al., 2024). The contribution here is the incumbent-relative pair of constraints, one on queue totals and one on individual decisions, and the like-for-like comparison it permits with compatibility rules.

2.1 A matched-budget guarantee

NFR interpolation forms probabilities

$$p^{\alpha}_{ik} = (1 - \alpha)p^{0}_{ik} + \alpha p^{1}_{ik},$$

where p^0 and p^1 are incumbent and candidate probabilities. Its hard assignment b^α traces a path from the incumbent to the raw candidate as α moves from zero to one. A deployment team can select a point that meets an NFR target while retaining validation accuracy. Queue Shift takes that point's movement and expected negative flips as its budgets.

Proposition 1 (Dominance at matched movement and flips)**.** *For any baseline assignment b and weights w, set $B = \mathcal{S}(b; m)$ and $C = F_w(b)$. Then $z^* = z^*(B, C)$ satisfies*

$$V_p(z^*) \geq V_p(b), \qquad \mathcal{S}(z^*; m) \leq \mathcal{S}(b; m), \qquad F_w(z^*) \leq F_w(b).$$

Proof. The baseline is feasible for Equation 1 by the definition of B and C. An optimum cannot have a lower objective value than a feasible comparison assignment, and every feasible assignment satisfies both budgets. □

The proof is one line; the content is in what is matched. With $w_i = p_{i,a_i}$, Queue Shift moves no more work between queues, risks no more expected negative flips, and has no lower expected accuracy than the baseline, all under the candidate's probabilities. Matching movement alone, the special case $C = \infty$, drops the protection an NFR rule exists to provide. In the simulations below, the movement-only solution keeps at least 73% of the raw candidate's negative-flip rate even at zero movement. A comparison at matched movement alone is therefore not like for like.

If p_{ik} is the true conditional probability of class k, $V_p(z)$ is conditional expected accuracy and F_w with $w_i = p_{i,a_i}$ is the conditional expected number of negative flips. Estimated probabilities require

more care. Let q_{ik} denote the true conditional probabilities, $e_{ik} = q_{ik} - p_{ik}$, and $D = \{i : z_i^* \neq b_i\}$. Cases outside D contribute the same error to both assignments, so

$$V_q(z^*) - V_q(b) \geq V_p(z^*) - V_p(b) - \frac{1}{n} \sum_{i \in D} \left(|e_{i,z_i^*}| + |e_{i,b_i}| \right). \tag{2}$$

The same argument bounds the flip budget's error. Each moved case i contributes q_{i,a_i} rather than p_{i,a_i} to the true expected flip load, so true expected negative flips exceed planned ones by at most $\sum_{i:z_i^* \neq a_i} |e_{i,a_i}|$. Neither bound can be computed in deployment, because q is unobserved, and both are loose in our simulations. Their use is diagnostic. Errors matter only on the cases the method moves, so the calibration worth checking is calibration on those cases. Replaying the solver on labeled history measures it directly.

2.2 Queue prices

The flow form has a second reading that matters for practice.

Proposition 2 (Queue prices)**.** *For any B and penalty $\lambda \geq 0$, there are prices $u_1, \ldots, u_K$ such that the penalized flow optimum assigns every case to a queue that maximizes*

$$p_{ik} - \lambda w_i \mathbf{1}\{k \neq a_i\} - u_k.$$

Proof. The prices are the optimal dual values (node potentials) of the queue nodes. Complementary slackness for minimum-cost flow requires every arc carrying flow to have zero reduced cost and every empty arc nonnegative reduced cost. For the arcs leaving case i, this is the stated maximization. □

The optimum is thus a per-queue threshold rule: subtract a price from each queue's probability and take the best queue. The price falls on probabilities, not logits. Ties among the maximizing queues occur only for marginal cases; in our checks they never involve more than one case per queue. The solver returns these prices.

Two consequences follow. First, prices learned on history and then applied case by case are the natural low-cost competitor. This is the primal-dual approach to online allocation, which learns dual prices on early arrivals and uses them on later ones (Devanur and Hayes, 2009; Agrawal et al., 2014), and it is how a routing system that cannot wait for a batch would deploy Queue Shift. Second, learned prices carry no guarantee for a given batch. Prices fitted to one mix of cases move the wrong amount of work when the next batch has a different mix. Section 3 measures how much this matters.

3 When the constraint binds

The propositions hold for any scores. Whether they matter depends on how far the candidate's predicted queue mix is from the incumbent's, how much of that difference the organization can absorb, and whether assignments can wait for a batch. This section names the settings where these conditions arise and uses simulation to show how large the effects can be. The simulations illustrate mechanisms; they do not estimate magnitudes for any organization.

Where queue totals are fixed in the short run. Queue Shift is useful only when work routed to a queue must be handled by staff assigned to that queue over the planning horizon. That describes support desks staffed by skill or product, content moderation queues organized by policy area or

language, insurance claims and fraud review tiers, and compliance alert review. In each, rosters are set days ahead and cross-training is partial. It does not describe a pool of interchangeable agents, where only total volume matters. Assignment must also be able to wait for a batch, such as an overnight backlog or a daily review queue. Live routing of one case at a time can use only the price rule of Proposition 2.

Mechanisms that move volume. Four scenarios make the candidate's predicted mix differ from the incumbent's in different ways. *Stable*: five queues with unequal base rates; the candidate adds an independent signal that sharpens every class. *Confused pair*: the incumbent barely separates the two largest queues, as with billing and technical tickets, and the candidate's new feature separates them; volume moves between those queues in a consistent direction. *Stale priors*: demand changed after the incumbent was trained, as after a product launch, and the candidate was trained on recent data; here the candidate's mix is closer to the truth, and holding the incumbent's totals preserves an outdated plan; the budget then limits how much of the needed change happens before staffing catches up. *Specialist queues*: fifteen queues with Zipf-distributed base rates, where small queues have little slack. Each scenario crosses four batch regimes: 2,000 or 300 cases with a fixed class mix, and 300 or 100 cases whose class mix is redrawn for every batch from a Dirichlet distribution around the base rates. Every scenario uses 30 independently trained pairs of multinomial logistic models, released only after the candidate wins on an independent 2,000-case sample. Raw candidate queue shift ranges from 7% to 25% of cases.

Matching flips changes the comparison. Figure 1 shows every method in the 300-case fixed-mix regime. Matched on queue shift alone (blue), Queue Shift appears to dominate interpolation by a wide margin at small budgets. Its negative-flip rate shows why. At zero movement, where interpolation keeps the incumbent and has no flips, movement-only Queue Shift flips 12–16% of the cases the incumbent got right, at least 73% of the raw candidate's rate. Matched on both queue shift and expected negative flips (green), the realized negative-flip rate stays within 0.7 points of interpolation's, and the true accuracy gain over interpolation is 1.0–2.2 points at $\alpha = 0.25$ and 0.3–0.8 points at $\alpha = 0.5$ across scenarios and batch regimes. At $\alpha = 0$ the flip budget is zero and both methods keep the incumbent. This like-for-like gain is smaller than the movement-only gain, and it is the relevant one when an NFR target is binding.

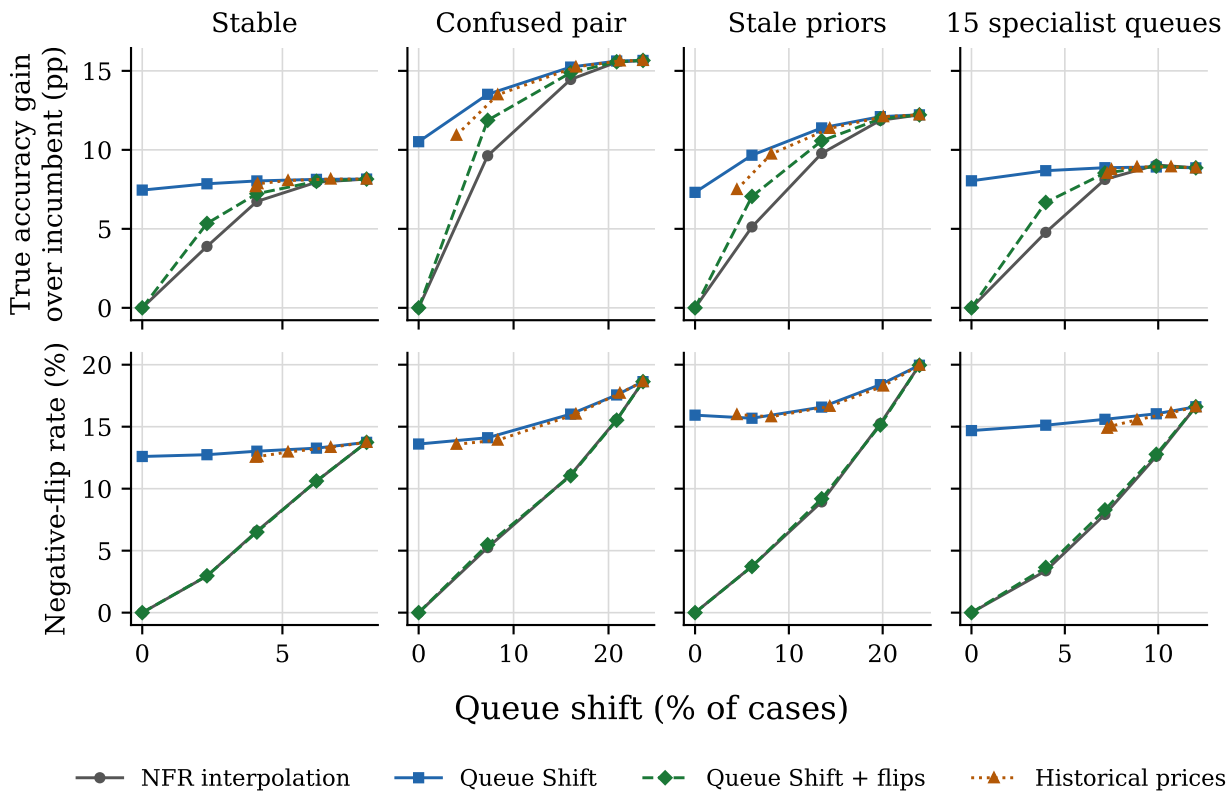


Figure 1: True accuracy gain over the incumbent (top) and realized negative-flip rate (bottom) against queue shift, 300-case batches with a fixed class mix. Interpolation and both Queue Shift arms are evaluated at interpolation's queue shift for $\alpha \in \{0, 0.25, 0.5, 0.75, 1\}$; historical prices are plotted at the queue shift they actually produced. Means across 30 trained model pairs.

What holding queue totals costs. Table 1 reports the accuracy lost by holding every queue total fixed, relative to deploying the raw candidate. The cost depends on the mechanism, not on the size of the update. When an update mostly reorders cases within a stable mix (the stable and specialist scenarios with fixed-mix batches), zero movement costs 0.3–0.8 points and keeps at least 91% of the update. When the update legitimately moves volume (confused pair and stale priors), it costs 4.7–6.5 points and keeps only 49–68%. Small, volatile batches raise the cost even for reordering updates, to 2.1–3.1 points, because the incumbent's totals on a small batch track noise as well as demand. The constraint is nearly free in the first case and a real tradeoff in the second, which is where a nonzero budget, chosen with the staffing plan, earns its keep. An earlier hypothesis that the matched-budget gain over interpolation grows with the raw candidate's queue shift is not supported (Figure A.3); the mechanism matters more than the size of the shift.

Table 1: Cost of holding every incumbent queue total

Scenario	Batch	Raw update (pp)	Cost of zero movement (pp) [95% CI]	Update kept (%)
Stable	2,000, fixed mix	8.12	0.51 [0.43, 0.58]	94
Stable	300, fixed mix	8.15	0.70 [0.57, 0.82]	91
Stable	300, volatile mix	8.16	1.24 [1.02, 1.46]	85
Stable	100, volatile mix	8.58	2.06 [1.58, 2.54]	76
Confused pair	2,000, fixed mix	15.45	4.97 [4.43, 5.51]	68
Confused pair	300, fixed mix	15.66	5.14 [4.61, 5.68]	67
Confused pair	300, volatile mix	15.52	5.45 [4.37, 6.53]	65
Confused pair	100, volatile mix	15.68	6.12 [5.24, 6.99]	61
Stale priors	2,000, fixed mix	12.13	4.71 [4.48, 4.94]	61
Stale priors	300, fixed mix	12.22	4.91 [4.64, 5.17]	60
Stale priors	300, volatile mix	11.74	5.14 [4.34, 5.94]	56
Stale priors	100, volatile mix	12.70	6.45 [5.71, 7.20]	49
15 specialist queues	2,000, fixed mix	9.02	0.34 [0.28, 0.40]	96
15 specialist queues	300, fixed mix	8.86	0.82 [0.72, 0.92]	91
15 specialist queues	300, volatile mix	9.15	2.06 [1.75, 2.37]	78
15 specialist queues	100, volatile mix	9.83	3.12 [2.67, 3.57]	68

Note: Raw update is the raw candidate's true accuracy gain over the incumbent. Cost of zero movement is the raw candidate's true accuracy minus that of Queue Shift at $B = 0$. True accuracy uses the simulation's conditional probabilities.

Prices versus exact assignment. Proposition 2 suggests a cheaper rule: learn queue prices once on history and apply them case by case. Table 2 compares this rule with exact per-batch assignment at interpolation's $\alpha = 0.5$ budget. The prices are close to optimal for the movement they produce, falling short of the exact assignment at that movement by at most 1.6 points anywhere in the design. But they do not control movement. At $\alpha = 0.5$ they exceed the batch's budget in 50–93% of batches, and at zero movement they move 2.0–12.3% of cases. The overshoot is largest for small, volatile batches and many small queues. The practical division is therefore clear. Where a budget must hold in every batch, solve each batch exactly. Where routing is online or the budget is a soft average, prices from Queue Shift are a sound approximation, and the exact solver supplies them.

Table 2: Historical queue prices versus exact per-batch assignment

Scenario	Batch	Budget (% cases)	Overshoot (pp of cases)	Over budget (% batches)	Value gap (pp)
Stable	2,000, fixed mix	3.2	0.72	77	0.03
Stable	300, fixed mix	4.1	1.11	75	0.04
Stable	300, volatile mix	4.5	0.96	68	-0.15
Stable	100, volatile mix	6.5	1.61	62	-0.14
Confused pair	2,000, fixed mix	15.7	0.15	55	0.02
Confused pair	300, fixed mix	16.0	0.55	53	0.07
Confused pair	300, volatile mix	15.8	0.31	54	-0.03
Confused pair	100, volatile mix	17.0	0.68	50	0.05
Stale priors	2,000, fixed mix	12.7	0.66	67	0.10
Stale priors	300, fixed mix	13.5	0.86	59	0.17
Stale priors	300, volatile mix	13.0	0.89	67	-0.02
Stale priors	100, volatile mix	14.3	1.54	62	0.01
15 specialist queues	2,000, fixed mix	4.6	1.40	93	0.03
15 specialist queues	300, fixed mix	7.1	1.72	82	0.01
15 specialist queues	300, volatile mix	7.7	2.26	92	-0.07
15 specialist queues	100, volatile mix	10.6	3.22	80	-0.04

Note: Budget is NFR interpolation's queue shift at $\alpha = 0.5$. Prices are learned once on a 2,000-case history at the same interpolation weight and applied case by case. Overshoot is the prices' extra queue shift; value gap is the true-accuracy shortfall against the exact assignment at the movement the prices actually used.

4 What a deployment would validate

A practical rollout separates model release from batch assignment. First, compare the raw candidate with the incumbent on later labeled data and reject candidates that do not improve the chosen predictive criterion. Second, measure the raw candidate's queue shift and the cost of holding queue totals on replayed history; if both are small, the constraint is nearly free and the budget can be tight. Third, choose the two budgets: queue movement from the staffing plan, and expected negative flips from an existing NFR target or the interpolation point that meets it. Fourth, solve Equation 1 on each planning batch. Finally, monitor queue loads, which hold by construction, and evaluate accuracy and negative flips when labels arrive.

The matched-budget guarantee needs no labels at assignment time, but its accuracy and flip interpretation needs useful probability scores on the cases the solver moves. Reliability plots restricted to moved cases, proper scoring rules, and out-of-time replay can check that condition.

Three limitations matter before claims become broad. First, workload is priced by queue averages; case-specific handling times and skill constraints within a queue are outside the model. Second, the evidence is synthetic, with correctly specified logistic models except where scores are rescaled. Third, the comparison is with one compatibility rule. A field study should report service times, queue-specific capacity, forecast horizon, and realized staffing actions, and should compare with the compatibility method the organization actually uses.

5 Conclusion

For routing classifiers, compatibility is partly an allocation problem. Negative flips protect individual decisions; queue shift protects the staffing plan. Queue Shift optimizes within both and, at the same movement and expected flips as any comparison assignment, has at least its expected accuracy under the candidate's probabilities. The flow optimum is a price rule, which gives online routing a principled approximation. The simulations locate where this matters. When an update reorders cases, holding queue totals is nearly free; when it moves volume, holding them is costly and the budget should be set together with staffing. The result is modest, exact, and testable.

References

Agrawal, S., Wang, Z., and Ye, Y. (2014). A dynamic near-optimal algorithm for online linear programming. *Operations Research*, 62(4):876–890.

Bansal, G., Nushi, B., Kamar, E., Weld, D. S., Lasecki, W. S., and Horvitz, E. (2019). Updates in human-AI teams: Understanding and addressing the performance/compatibility tradeoff. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 33, pages 2429–2437.

Devanur, N. R. and Hayes, T. P. (2009). The adwords problem: Online keyword matching with budgeted bidders under random permutations. In *Proceedings of the 10th ACM Conference on Electronic Commerce*.

Elmachtoub, A. N. and Grigas, P. (2022). Smart "predict, then optimize". *Management Science*, 68(1):9–26.

Jiang, H., Narasimhan, H., Bahri, D., Cotter, A., and Rostamizadeh, A. (2022). Churn reduction via distillation. In *International Conference on Learning Representations*.

Milani Fard, M., Cormier, Q., Canini, K., and Gupta, M. (2016). Launch and iterate: Reducing prediction churn. In *Advances in Neural Information Processing Systems*, volume 29.

Rabkin, L., Cohen, I., and Singer, G. (2024). Resource allocation in ordinal classification problems: A prescriptive framework utilizing machine learning and mathematical programming. *Engineering Applications of Artificial Intelligence*, 132:107914.

Srivastava, M., Nushi, B., Kamar, E., Shah, S., and Horvitz, E. (2020). An empirical analysis of backward compatibility in machine learning systems. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 3272–3280.

Tai, K. S., Bailis, P. D., and Valiant, G. (2021). Sinkhorn label allocation: Semi-supervised classification via annealed self-training. In *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pages 10065–10075.

Träuble, F., von Kügelgen, J., Kleindessner, M., Locatello, F., Schölkopf, B., and Gehler, P. (2021). Backward-compatible prediction updates: A probabilistic approach. In *Advances in Neural Information Processing Systems*, volume 34.

Watson-Daniels, J., Calmon, F. d. P., D'Amour, A., Long, C., Parkes, D. C., and Ustun, B. (2024). Predictive churn with the set of good models.

Appendix

A.1 Estimated-model validation

This experiment stress-tests probability quality in the stable five-queue setting. The incumbent is a multinomial logistic regression fitted on one signal block; the candidate is fitted on that block plus an independent innovation block. Scenarios vary historical sample size, innovation strength, and candidate-score temperature, which changes the scores used by the solver without changing hard predictions. Each scenario has 50 trained pairs released after an independent accuracy win, four 300-case planning batches per pair, and uncertainty computed across pairs. The full design is in `validation_plan.md`. Table A.1 reports both Queue Shift arms. Candidate updates pass the release gate in at least 88% of trained pairs.

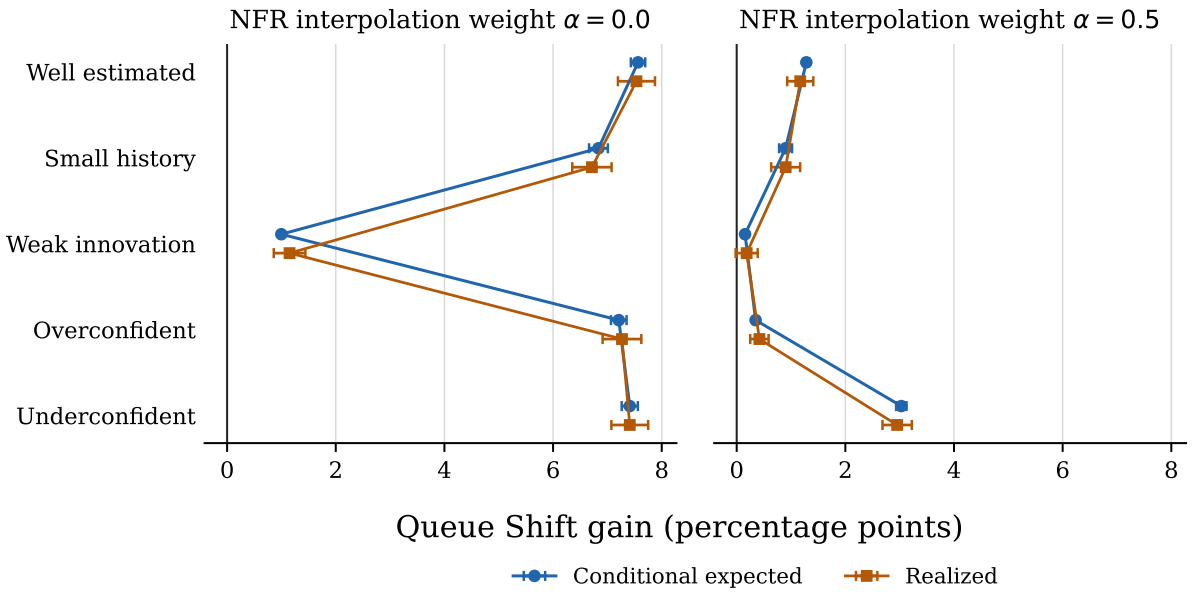


Figure A.1: Movement-only Queue Shift gain over NFR interpolation at matched queue movement. Points are means across independently trained, accepted model updates; bars are 95% normal intervals. Conditional expected accuracy uses the simulation's true probabilities.

Table A.1: Estimated-model validation against NFR interpolation

		Shift	NFR (%)			True gain over interp. (pp) [95% CI]	
Scenario	α	(%)	Interp.	QS	QS+flips	QS	QS+flips
Well estimated	0.0	0.00	0.0	12.3	0.0	7.56 [7.43, 7.70]	0.00 [0.00, 0.00]
Well estimated	0.5	3.82	6.7	12.7	6.8	1.28 [1.23, 1.33]	0.50 [0.47, 0.52]
Well estimated	1.0	7.42	13.4	13.4	13.4	0.00 [0.00, 0.00]	0.00 [0.00, 0.00]
Small history	0.0	0.00	0.0	13.4	0.0	6.84 [6.66, 7.01]	0.00 [0.00, 0.00]
Small history	0.5	4.11	7.5	14.0	7.7	0.90 [0.78, 1.02]	0.36 [0.32, 0.41]
Small history	1.0	7.88	14.9	14.9	14.9	0.00 [0.00, 0.00]	0.00 [0.00, 0.00]
Weak innovation	0.0	0.00	0.0	8.0	0.0	1.00 [0.96, 1.04]	0.00 [0.00, 0.00]
Weak innovation	0.5	1.89	4.0	8.1	4.1	0.15 [0.13, 0.18]	0.03 [0.02, 0.05]
Weak innovation	1.0	3.02	8.3	8.3	8.3	0.00 [0.00, 0.00]	0.00 [0.00, 0.00]
Overconfident	0.0	0.00	0.0	12.4	0.0	7.21 [7.07, 7.35]	0.00 [0.00, 0.00]
Overconfident	0.5	5.42	9.4	12.9	9.5	0.35 [0.31, 0.38]	0.15 [0.13, 0.17]
Overconfident	1.0	7.58	13.3	13.3	13.3	0.00 [0.00, 0.00]	0.00 [0.00, 0.00]
Underconfident	0.0	0.00	0.0	12.3	0.0	7.41 [7.26, 7.56]	0.00 [0.00, 0.00]
Underconfident	0.5	2.77	3.8	12.5	3.5	3.03 [2.93, 3.13]	0.84 [0.80, 0.88]
Underconfident	1.0	7.45	13.4	13.4	13.4	0.00 [0.00, 0.00]	0.00 [0.00, 0.00]

Note: Multinomial logistic models are estimated on historical samples and released only after the candidate wins on an independent labeled sample. Each row compares exact assignment with probability interpolation at the same realized queue shift (QS), and at both its queue shift and its expected negative flips (QS+flips). True gain uses the simulation's conditional probabilities and is unavailable in applications. Results average four planning batches within each independently trained accepted update before averaging across updates.

A.2 Oracle-posterior experiment

As a mechanical check, an oracle experiment supplies the exact incumbent and candidate posteriors from the same stable Gaussian population. Figure A.2 and Table A.2 show the full matched path. At zero movement, movement-only Queue Shift gains 7.64 points of realized accuracy over the incumbent but flips 12.3% of the cases the incumbent got right, where interpolation flips none. At $\alpha = 0.2$,

matched on both queue shift and expected flips, Queue Shift flips 2.3% against interpolation's 2.3% and gains 1.62 points.

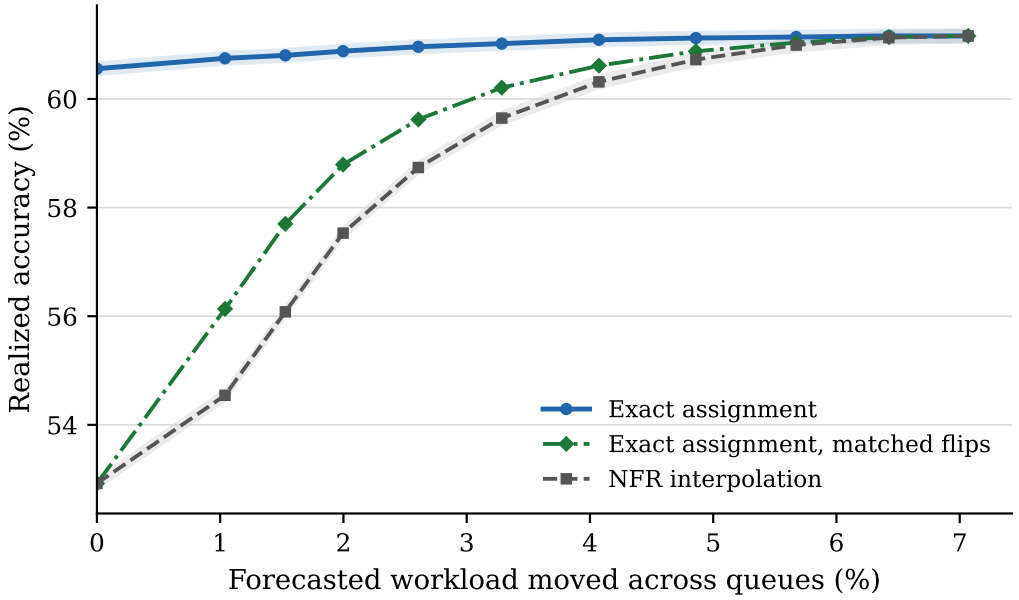


Figure A.2: Oracle-posterior comparison at matched constraints. Bands are 95% normal intervals across independently generated updates.

Table A.2: NFR interpolation versus exact assignment at matched constraints

	Queue	NFR (%)			Accuracy (%)		
α	shift (%)	Interp.	QS	QS+flips	Interp.	QS	QS+flips
0.0	0.00	0.00	12.28	0.00	52.92	60.56	52.92
0.2	1.53	2.30	12.38	2.29	56.08	60.80	57.70
0.5	3.28	6.49	12.56	6.47	59.65	61.02	60.21
0.8	5.67	10.79	12.98	10.76	60.99	61.14	61.04
1.0	7.07	13.28	13.28	13.28	61.16	61.16	61.16

Note: The data-generating process is fixed across release and planning samples. The candidate observes an additional independent signal and is deployed only after beating the incumbent on an independent out-of-sample release set. Each row compares probability interpolation (Interp.) with exact assignment under the candidate posterior at the interpolation point's queue shift (QS), and at both its queue shift and its expected negative flips (QS+flips). Results average 1,000 planning batches from 200 accepted updates. Accuracy and NFR use labels revealed only after every assignment is fixed.

A.3 Raw queue shift and the matched-budget gain

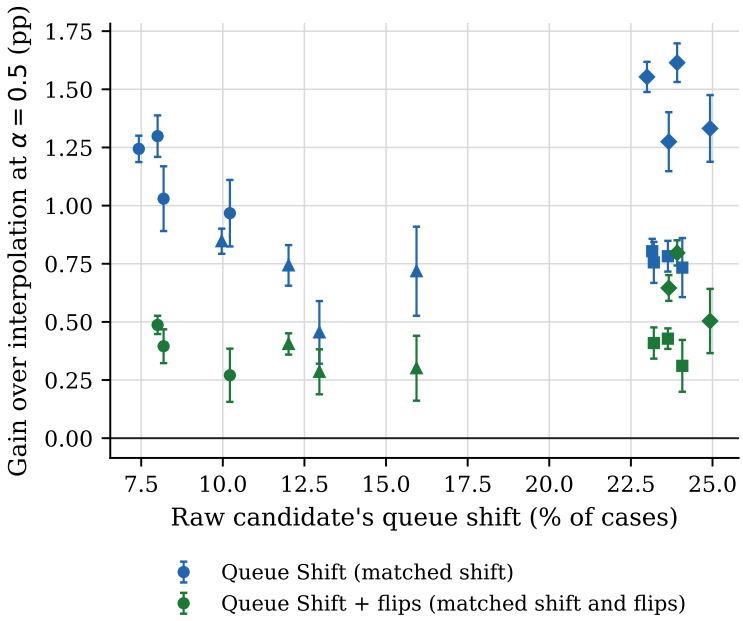


Figure A.3: Gain over interpolation at $\alpha = 0.5$ against the raw candidate's queue shift. Each point is one scenario and batch regime (marker shape gives the scenario); bars are 95% normal intervals across trained pairs. The joint arm is not computed for 2,000-case batches. The gain does not rise with the raw shift.